Mesclagem Laplaciana de Texturas Adaptativa por Distância com *Level Skipping* em GPU

Ícaro Vinícius Vinícius Macedo Caio Dias
Instituto Federal do Norte de Minas Gerais – IFNMG, Campus Montes Claros
{icaro.vinicius, vinicius.macedo, caio.dias}@aluno.ifnmg.edu.br

Resumo. A mesclagem linear ponto-a-ponto de texturas introduz perda de até 50% da variância estatística no ponto médio da transição, gerando perda de contraste e artefatos de sobreposição fantasma (*ghosting*). Recentemente, Wronski (2025) propôs decompor o sinal em pirâmides Laplacianas diretamente no *fragment shader* reutilizando a cadeia de *mipmaps* da GPU, mas com custo fixo de 21 leituras de textura por pixel ($K = 6$ níveis). Este trabalho investiga duas otimizações para reduzir a carga sobre as Unidades de Mapeamento de Textura (TMUs): (1) *Level Skipping*, que amostra apenas níveis alternados da cadeia de *mipmaps* mantendo a soma telescópica exata; e (2) *LOD-Adaptive Blending*, que adapta a profundidade da pirâmide pela distância e derivadas de tela. Resultados preliminares demonstram redução de 42,9% nas leituras (12 *fetches*/pixel) com PSNR = 54,64 dB no *Level Skipping* Puro, superando o truncamento simples de mesmo custo em +21,77 dB.

Palavras-chave: Computação Gráfica; Mesclagem de Texturas; Pirâmide Laplaciana; Mipmapping; Nível de Detalhe (LOD).

Abstract. Pointwise linear texture blending reduces local statistical variance by up to 50% at the transition midpoint, causing contrast loss and ghosting artifacts. Recently, Wronski (2025) proposed a buffer-free Laplacian pyramid blending approach within a single fragment shader using hardware mipmap chains, at a uniform cost of 21 texture fetches per pixel ($K = 6$ levels). This work investigates two optimizations to reduce Texture Mapping Unit (TMU) bandwidth: (1) *Level Skipping*, which samples alternating mipmap levels while preserving exact telescopic reconstruction; and (2) *LOD-Adaptive Blending*, which scales pyramid depth according to viewing distance and screen-space derivatives. Preliminary results show a 42.9% reduction in texture fetches (12 fetches/pixel) with 54.64 dB PSNR for Pure Level Skipping, outperforming simple truncation of equal cost by +21.77 dB.

Keywords: Computer Graphics; Texture Blending; Laplacian Pyramid; Mipmapping; Level of Detail (LOD).

1 Introdução

Orientação aos autores: Contextualize o problema de Computação Gráfica, explique sua relevância, apresente o trabalho de referência e conduza o leitor até a questão específica investigada. A introdução deve terminar com a questão/hipótese e as contribuições do artigo.

A composição espacial de materiais em terrenos virtuais e superfícies PBR depende de operadores de mesclagem controlados por uma máscara $\alpha \in [0,1]$. A interpolação linear padrão ($\mathrm{mix}(I_1, I_2, \alpha)$) impõe um compromisso severo: transições estreitas revelam costuras visíveis (*seams*), ao passo que transições largas atenuam altas frequências e produzem efeito fantasma (*ghosting*) (Heitz and Neyret, 2018).

Para contornar esse problema em tempo real sem alocar buffers extras, Wronski (2025) propôs aproximar a mesclagem em pirâmides Laplacianas de Burt and Adelson (1983) dentro do *fragment shader* usando a cadeia de *mipmaps* de hardware. Contudo, a avaliação completa de $K = 6$ níveis requer 21 instruções de leitura de textura (*texture fetches*) por fragmento, o que sobrecarrega as Unidades de Mapeamento de Textura (TMUs), especialmente em cenas 3D extensas.

Questão de pesquisa e contribuições (Esqueleto – Checkpoint 2): Investigamos se é possível reduzir o tráfego de textura da mesclagem Laplaciana em GPU via salto de níveis (*Level Skipping*) e adaptação por nível de detalhe (*LOD-Adaptive Blending*) sem perda perceptível de qualidade. Este trabalho entrega:

- Reprodução da baseline linear e do método de Wronski (2025) em Python (`float32`) e WebGL 2.0 / Three.js;
- Formulação algébrica do *Level Skipping* Puro ($\{0,2,4,6\}$) e Híbrido ($\{0,1,3,5\}$) com soma telescópica preservada;
- Esquema de mesclagem adaptativa por distância com transição hermítica (`smoothstep`) livre de *LOD popping*;
- *[Em andamento para a Entrega Final]* Avaliação em cenário 3D completo com perspectiva e medição de *frame time* em GPU.

2 Trabalho de Referência e Trabalhos Relacionados

Orientação aos autores: Apresente o artigo-base com precisão suficiente para que o leitor entenda a técnica que será modificada. Discuta apenas a literatura necessária para posicionar a proposta. Verifique trabalhos posteriores antes de afirmar novidade.

Dadas duas texturas independentes modeladas como variáveis aleatórias X e Y de mesma variância σ^2 e $\mathrm{Cov}(X,Y) = 0$, a combinação convexa linear no ponto médio ($\alpha = 0{,}5$) resulta em:

$$\mathrm{Var}(0{,}5X + 0{,}5Y) = (0{,}25 + 0{,}25)\sigma^2 = 0{,}50\,\sigma^2, \tag{1}$$

destruindo 50% da variância original (Heitz and Neyret, 2018; Burley, 2019; Mikkelsen, 2022).

No artigo-base, Wronski (2025) utiliza os níveis de *mipmap* G_k ($k \in \{0,\ldots,K\}$) como pirâmide Gaussiana e extrai bandas Laplacianas por Diferença de Gaussianas (DoG):

$$L_k = G_k - G_{k+1}. \tag{2}$$

A reconstrução apoia-se na identidade da soma telescópica:

$$(G_0 - G_1) + (G_1 - G_2) + \cdots + (G_{K-1} - G_K) + G_K = G_0. \tag{3}$$

[A expandir na Entrega Final: detalhamento comparativo com operadores de preservação de histograma, Mix-Max (Fournier and Sauvage, 2024) e edição no domínio de Poisson (Pérez et al., 2003).]

3 Proposta

Orientação aos autores: Descreva exatamente o que foi alterado, acrescentado ou investigado. Diferencie o método original da contribuição da equipe. Use diagramas, pseudocódigo ou equações quando melhorarem a reprodutibilidade.

3.1 Salto de Níveis (*Level Skipping*)

Para reduzir o custo de 21 para 12 leituras por pixel (−42,9%) sem truncar o alcance espectral da pirâmide em $K = 3$, propomos amostrar apenas níveis alternados ($\Delta k = 2$) no subconjunto par $\mathscr{S}_{\text{puro}} = \{0,2,4,6\}$:

$$\widehat{L}_k = G_k - G_{k+2}, \quad k \in \{0,2,4\}. \tag{4}$$

A soma telescópica permanece algebricamente exata fora da zona de transição:

$$(G_0 - G_2) + (G_2 - G_4) + (G_4 - G_6) + G_6 = G_0. \tag{5}$$

Investigamos também a variante *Híbrida* $\mathscr{S}_{\text{híb}} = \{0,1,3,5\}$, que preserva a primeira banda fina $G_0 - G_1$ intacta.

3.2 Mesclagem Adaptativa por Distância (*LOD-Adaptive*)

Particionamos o domínio espacial em três regimes operacionais conforme o indicador composto de distância/LOD $\xi \in [0,1]$ obtido via derivadas de tela (`dFdx`, `dFdy`):

1. **Zona Próxima** (21 ***taps***)**:** Pirâmide completa ($K = 6$);
2. **Zona Intermediária** (12 ***taps***)**:** *Level Skipping* ($\Delta k = 2$);
3. **Zona Distante** (3 ***taps***)**:** *Linear Blending* no nível de *mipmap* corrente.

As fronteiras entre regimes são interpoladas sobre os pesos da máscara via smoothstep, garantindo continuidade visual sem *popping*.

4 Implementação

Orientação aos autores: Informe apenas os detalhes necessários para compreender e reproduzir o trabalho: linguagem, bibliotecas, versão da baseline, alterações relevantes, hardware e parâmetros. O manual completo de instalação deve ficar no README do GitHub.

A implementação divide-se em dois módulos no repositório:

- **Bancada Offline (Python 3.10+):** `src/laplacian_blend.py`, operando em precisão `float32` com `NumPy`, `Pillow` e `SciPy`;
- **Visualizador Interativo (WebGL 2.0 / Three.js):** `demo_webgl/main.js`, contendo os *shaders* GLSL, HUD de contagem de *texture fetches* e mapa de calor (*Debug Heatmap*).

Listing 1: Trecho estrutural do cálculo de bandas com *Level Skipping* e transição suave no *fragment shader* GLSL.

```
// Pesos de ativacao continua para evitar popping entre zonas
float wHigh = 1.0 - smoothstep(0.18, 0.42, xi);
float wMid = 1.0 - smoothstep(0.42, 0.72, xi);

// Bandas Laplacianas com salto de nivel (Delta k = 2)
vec3 L0 = mix(g2_0 - g2_2, g1_0 - g1_2, mix(a2, a0, wHigh));
vec3 L2 = mix(g2_2 - g2_4, g1_2 - g1_4, mix(a4, a2, wMid));
vec3 L4 = mix(g2_4 - g2_6, g1_4 - g1_6, a4);
vec3 G6 = mix(g2_6, g1_6, a6);
vec3 blended = G6 + L4 + L2 + L0;
```

5 Metodologia Experimental

Orientação aos autores: Explique como a hipótese será testada. Identifique baseline, dados/cenas, parâmetros, métricas, quantidade de execuções, sementes e regras de comparação. Baseline e proposta devem ser avaliadas sob condições equivalentes.

5.1 Questão experimental

Verificar se o *Level Skipping* supera o truncamento direto da pirâmide para o mesmo orçamento de 12 leituras/pixel e avaliar o comportamento do método adaptativo em 2D e 3D.

5.2 Baseline e variantes

São comparados: (i) *Linear Blending* (3 *fetches*); (ii) NVIDIA Completo $N = 6$ (21 *fetches*, referência); (iii) NVIDIA Truncado $N \in \{3,4,5\}$; (iv) *Level Skipping* Puro e Híbrido (12 *fetches*); e (v) Adaptativo por Distância.

5.3 Dados, modelos ou cenas

Texturas de albedo de terreno (1024×1024, rocha vs. musgo com máscara de Perlin) e mapas de normais PBR (512×512) do conjunto suplementar de Wronski (2025).

5.4 Métricas e protocolo

Contagem de *fetches*/pixel, PSNR (dB), SSIM, variância local na zona de transição ($0{,}05 < \alpha < 0{,}95$), taxa de *overshoot* e erro angular médio em graus (para normais).

6 Resultados

Orientação aos autores: Apresente resultados quantitativos e visuais. Toda tabela e figura deve ser citada no texto e responder a uma questão experimental. Evite imagens meramente decorativas.

A Tabela 1 apresenta os resultados quantitativos preliminares obtidos no Checkpoint 2 para o Caso (a) – Terreno Albedo e para o Caso (c2) – Mapas de Normais.

Tabela 1: Resultados preliminares (Checkpoint 2): comparação entre baseline, truncamento e propostas em precisão `float32`.

Método	Fetches	Red. TMU	PSNR (dB)	SSIM	Erro Normal
Linear Blend (Base)	3,00	−85,7%	32,21	0,9957	1,105°
NVIDIA Truncado ($N = 3$)	12,00	−42,9%	32,87	0,9967	0,801°
NVIDIA Completo ($N = 6$)	21,00	0,0%	∞	1,0000	0,000°
Skip Puro $\{0,2,4,6\}$	**12,00**	**−42,9%**	**54,64**	**1,0000**	**0,331°**
Skip Híbrido $\{0,1,3,5\}$	12,00	−42,9%	37,62	0,9990	0,466°
Adaptativo por Distância	13,98	−33,4%	38,69	0,9989	0,436°

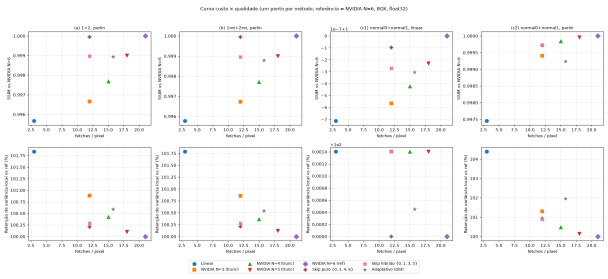


Figura 1: Fronteira de Pareto preliminar (Fetches/px vs. PSNR) gerada nos experimentos do Checkpoint 2.

A Figura 1 apresenta a curva de Pareto preliminar relacionando custo de amostragem e PSNR.

[A expandir na Entrega Final: inserção da tabela de tempo de renderização em milissegundos via `EXT_disjoint_timer_query_webgl2` em 1080p/4K e painel visual da cena 3D em perspectiva.]

7 Discussão

Orientação aos autores: Interprete os resultados. Discuta ganhos, perdas, falhas, compromissos, limitações, ameaças à validade e o que ainda precisaria ser testado antes de uma submissão.

Síntese preliminar e dificuldades do Checkpoint 2:

- ***Level Skipping* vs. Truncamento** (12 ***fetches***)**:** O *Skip* Puro supera o truncamento $N = 3$ em $+21{,}77$ dB de PSNR porque alcança o nível G_6, preservando a transição suave de baixa frequência;
- **Limitação do teste 2D plano para o método Adaptativo:** Em uma imagem 2D ortográfica (1 pixel = 1 texel), degradar para *Linear Blend* no topo da tela descarta altas frequências ainda visíveis. Em perspectiva 3D real, a minificação óptica filtra naturalmente essas frequências (limite de Nyquist);
- **Principais dificuldades tratadas:** (i) ausência de `textureQueryLod` no WebGL 2.0, contornada via Jacobiano (`dFdx`/`dFdy`); e (ii) eliminação de *LOD popping* interpolando os pesos da máscara em vez de escalar as bandas diretamente.

[A expandir na Entrega Final: discussão da anisotropia em ângulos rasantes e análise de sensibilidade aos filtros Box, Bilinear, Bicúbico e Lanczos.]

8 Conclusão

Orientação aos autores: Retome o problema, sintetize a contribuição e os resultados mais importantes. Não introduza novos experimentos. Indique trabalhos futuros concretos.

[Seção em elaboração para a Entrega Final (29/11/2026).]

Plano objetivo de conclusão:

1. Consolidar o cenário 3D em perspectiva para validação empírica da minificação óptica no modo adaptativo;
2. Coletar métricas de *frame time* em GPU (`EXT_disjoint_timer_query_webgl2`);
3. Expandir a discussão experimental, remover as caixas de orientação (`\mostrarorientacoesfalse`) e finalizar o artigo entre 8 e 12 páginas.

Referências

Brent Burley. On histogram-preserving blending for randomized texture tiling. *Journal of Computer Graphics Techniques (JCGT)*, 8(4):31–53, 2019. URL `http://jcgt.org/published/0008/04/02/`.

Peter J. Burt and Edward H. Adelson. The Laplacian pyramid as a compact image code. *IEEE Transactions on Communications*, 31 (4):532–540, 1983. doi: 10.1109/TCOM.1983.1095851.

Nicolas Fournier and Basile Sauvage. Mix-max: A pointwise texture mixing operator. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 7(1):1–18, 2024. doi: 10.1145/3651289.

Eric Heitz and Fabrice Neyret. High-performance by-example noise using a histogram-preserving blending operator. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 1 (2):1–25, 2018. doi: 10.1145/3233304.

Morten S. Mikkelsen. Practical real-time hex-tiling. *Journal of Computer Graphics Techniques (JCGT)*, 11(2):77–94, 2022. URL `http://jcgt.org/published/0011/02/05/`.

Patrick Pérez, Michel Gangnet, and Andrew Blake. Poisson image editing. *ACM Transactions on Graphics (TOG)*, 22(3):313–318, 2003. doi: 10.1145/882262.882269.

Bartlomiej Wronski. GPU-friendly Laplacian texture blending. *Journal of Computer Graphics Techniques (JCGT)*, 14(1):21–39, 2025. ISSN 2331-7418. URL `http://jcgt.org/published/0014/01/02/`.